LEVELUP CLOUD TRAININGS

PySpark RDD Masterclass

Module 6 — Student Class Notes

saveAsTextFile • coalesce • repartition • foreach • pipe • join • union • mapPartitions

Trainer: Sateesh Pabbathi | Microsoft MVP | MCT | 22+ Years Experience | 5000+ Students

Module 6 — Agenda & Overview

#	Topic	Type	Difficulty
1	saveAsTextFile	Action	Easy
2	coalesce	Transformation	Easy
3	repartition	Transformation	Medium
4	foreach / foreachPartition	Action	Medium
5	pipe	Transformation	Medium
6	join / leftOuterJoin / rightOuterJoin / fullOuterJoin	Transformation	Medium
7	union	Transformation	Easy
8	mapPartitions / mapPartitionsWithIndex	Transformation	Hard

What You Will Learn in This Module

This module covers the essential RDD output operations (how to write data), partition management operations (how to control data distribution), advanced transformations (mapPartitions for performance), and multi-dataset operations (join, union). By the end, you will be able to build production-grade PySpark pipelines used at companies like Flipkart, Swiggy, HDFC Bank, and TCS.

1. saveAsTextFile

What is saveAsTextFile?

saveAsTextFile is an ACTION in PySpark that saves the contents of an RDD to a text file on a distributed file system such as HDFS, S3, or local disk. Each element of the RDD is converted to a string and written as one line in the output file.

> **Real-World Analogy — TCS Nightly Report**
> Imagine a TCS data pipeline that processes millions of employee payroll records every month-end. After all calculations are done in memory (RDD transformations), saveAsTextFile is the final step — writing the results to HDFS so the finance system can pick them up. Just like hitting 'Save' after editing a large spreadsheet.

How Output Files Are Created

saveAsTextFile creates a DIRECTORY (not a single file) at the specified path. Inside this directory, each partition produces one part file. An RDD with 4 partitions creates 4 files:

```
/output/eod_report/
    _SUCCESS          ← created when job completes successfully
    part-00000        ← data from partition 0
    part-00001        ← data from partition 1
    part-00002        ← data from partition 2
    part-00003        ← data from partition 3
```

Syntax & Basic Usage

```
from pyspark import SparkContext
sc = SparkContext('local', 'SaveDemo')

# Sample: Flipkart daily order summary
orders = sc.parallelize([
    'ORD001,Hyderabad,Electronics,5000',
    'ORD002,Mumbai,Clothing,1200',
    'ORD003,Delhi,Books,450',
    'ORD004,Chennai,Groceries,890'
])

# Save to local path (4 records → 1 part file since 1 partition by default here)
orders.saveAsTextFile('/tmp/flipkart_orders')

# Save to HDFS
orders.saveAsTextFile('hdfs://namenode:9000/data/orders/2024-01-15')

# Save with GZIP compression (production best practice for large data)
from pyspark.hadoop.io.compress import GzipCodec
orders.saveAsTextFile('/tmp/orders_compressed', GzipCodec)
# Creates: part-00000.gz  part-00001.gz  ...
```

Controlling Output File Count

By default, the number of output files = number of partitions. Use coalesce() or repartition() BEFORE saveAsTextFile to control file count:

```
# Problem: RDD has 500 partitions → 500 output files (too many small files!)
big_rdd = sc.textFile('/data/raw/*.csv')  # may create 500 partitions
print(big_rdd.getNumPartitions())         # e.g., 500

# Solution: coalesce before saving
big_rdd.coalesce(10).saveAsTextFile('/data/output/daily_report')
# Now creates exactly 10 part files

# Single file output (use carefully — bottleneck for huge data)
big_rdd.coalesce(1).saveAsTextFile('/data/output/single_file')
```

Critical Production Rules

```
# ❌PROBLEM: saveAsTextFile FAILS if output directory already exists
orders.saveAsTextFile('/data/output')  # Run 1: OK
orders.saveAsTextFile('/data/output')  # Run 2: ERROR! Directory exists

# ✅FIX 1: Use timestamped paths
from datetime import datetime
path = f'/data/output/{datetime.now().strftime("%Y%m%d_%H%M%S")}'
orders.saveAsTextFile(path)

# ✅FIX 2: Delete directory before saving (using Hadoop FS API)
hadoopConf = sc._jsc.hadoopConfiguration()
fs = sc._jvm.org.apache.hadoop.fs.FileSystem.get(hadoopConf)
output_path = sc._jvm.org.apache.hadoop.fs.Path('/data/output')
if fs.exists(output_path):
    fs.delete(output_path, True)  # True = recursive delete
orders.saveAsTextFile('/data/output')

# ✅FIX 3: Use DataFrame write API (supports overwrite mode)
df.write.mode('overwrite').text('/data/output')
```

> **Sateesh Says**
>
> *In every real project at TCS, Infosys, or Wipro, you will ALWAYS see saveAsTextFile fail on the second run because the directory exists. The fix is simple — either delete first, or use timestamped paths. Production jobs always use date-partitioned HDFS paths like /data/orders/year=2024/month=01/day=15 so each daily run writes to a fresh path.*

✅Key Points to Remember

- saveAsTextFile is an ACTION — it triggers execution of the entire DAG
- Creates a DIRECTORY with part-XXXXX files (one per partition)
- Fails if output directory already exists — handle this in production code
- Each element is converted to string using .toString() — format data BEFORE saving
- Use coalesce() before saving to control number of output files
- GzipCodec compression reduces file size by 70-80% for text data

- DataFrame .write.mode('overwrite').text() is preferred for overwrite support

2. coalesce Transformation

What is coalesce?

coalesce(n) is a transformation that reduces the number of partitions in an RDD to n WITHOUT triggering a full shuffle. It works by merging nearby partitions on the same executor node — local data movement only. This makes it faster than repartition for reducing partition count.

> **Real-World Analogy — ICICI Bank Branch Merger**
>
> ICICI Bank has 100 branches in Hyderabad processing transactions separately. At end of day they want to consolidate into 5 regional hubs. Instead of moving everyone to a central office and redistributing (that's repartition), they merge nearby branches — Madhapur + Hitech City become one hub, Ameerpet + Punjagutta become another. Minimum travel, same result. That's coalesce.

Syntax & Usage

```
# Create RDD with 20 partitions
data = sc.parallelize(range(1, 1001), 20)
print(f'Initial: {data.getNumPartitions()}')  # 20

# Reduce to 5 partitions — NO full shuffle
reduced = data.coalesce(5)
print(f'After coalesce(5): {reduced.getNumPartitions()}')  # 5

# □ coalesce CANNOT increase partitions
attempted_increase = data.coalesce(100)  # Will stay at 20!
print(f'After coalesce(100): {attempted_increase.getNumPartitions()}')  # Still 20

# Common pattern: reduce before writing
processed_rdd = sc.textFile('/data/raw/*').map(process).filter(validate)
processed_rdd.coalesce(10).saveAsTextFile('/data/output')
```

When to Use coalesce

Situation	Use coalesce?	Reason
After heavy filter (much less data)	✅Yes	Data shrank — reduce partition overhead
Before saveAsTextFile (control file count)	✅Yes	Fewer, larger output files
Before join (need more partitions)	❌No — use repartition	coalesce cannot increase
Data is skewed (uneven partition sizes)	❌No — use repartition	coalesce won't rebalance
Reducing from 1000 to 5 partitions	✅Yes (if skew OK)	Faster — no full shuffle

Sateesh Says

Real production scenario at every company: A job starts with 500 partitions (from a big input file), then a filter keeps only 5% of records. Now you have 500 near-empty partitions! Before any further processing, call coalesce(20) to consolidate. Otherwise Spark wastes time scheduling 500 tasks that each process 10 records. This single line of code can cut job time by 50%.

✅Key Points to Remember

- coalesce(n) can only DECREASE partitions (or keep same) — never increase
- Does NOT trigger a full shuffle — merges partitions locally (fast)
- May create uneven partition sizes (skew) — acceptable for final output stages
- Always verify with .getNumPartitions() after calling coalesce()
- Best used: after filter operations and before saveAsTextFile

3. repartition Transformation

What is repartition?

repartition(n) is a transformation that FULLY redistributes data across n new partitions using a shuffle. Unlike coalesce, it can both increase and decrease partition count, and it guarantees even data distribution across all partitions.

> **Real-World Analogy — Swiggy Delivery Zone Rebalancing**
> Swiggy assigns orders to delivery partners by area. If one zone (Hitech City) gets 40% of all orders and others get 2-3% each, partners in Hitech City are overwhelmed while others are idle. Repartition is like Swiggy HQ reshuffling ALL orders and redistributing evenly across all zones — full redistribution, perfectly balanced. Yes it takes effort (shuffle), but the result is fair.

Syntax & Usage

```
# Start with small RDD — 5 partitions
sales = sc.parallelize(flipkart_sales_data, 5)
print(sales.getNumPartitions())  # 5

# Increase to 200 partitions for parallel processing
sales_parallel = sales.repartition(200)
print(sales_parallel.getNumPartitions())  # 200

# Decrease with even distribution (repartition balances; coalesce may not)
sales_reduced = sales_parallel.repartition(20)
print(sales_reduced.getNumPartitions())  # 20  (evenly distributed)

# Check distribution is even
counts = sales_reduced.mapPartitions(lambda p: [sum(1 for _ in p)]).collect()
print(counts)  # Should be roughly equal numbers
```

When to Use repartition

- Before join operations — ensure both RDDs have balanced partitions on the join key
- When incoming data has partition skew — redistribute evenly before heavy computation
- After reading from a small file — increase parallelism for processing
- Before groupBy/aggregation — ensure even distribution to avoid straggler tasks

coalesce vs repartition — Full Comparison

Property	coalesce(n)	repartition(n)
Full Shuffle	No (local merge only)	Yes (full data movement)
Can Increase Partitions	No — silently stays at current	Yes
Can Decrease Partitions	Yes (primary use case)	Yes

Data Distribution	May be uneven (skewed)	Always even
Speed	Faster (no shuffle)	Slower (shuffle)
Network Transfer	Minimal	High
Best Used For	Reducing before write	Increasing / rebalancing
Use Case Example	coalesce(5) before saveAsTextFile	repartition(200) before big join

Sateesh Says

The most common interview question: 'When do you use coalesce vs repartition?' Short answer: Use coalesce to shrink (faster, no shuffle), use repartition to grow or rebalance (slower but even). Pro tip: After a 90% filter, use coalesce. Before a 10TB join, use repartition. These two decisions alone account for 50% of Spark performance tuning.

✅Key Points to Remember

- repartition(n) triggers FULL SHUFFLE — all data moves across network
- Both INCREASE and DECREASE of partition count is supported
- Always produces evenly distributed partitions (no skew)
- Use before joins, aggregations, and when you need to increase parallelism
- spark.sql.shuffle.partitions (default 200) controls post-shuffle partition count for DataFrames

4. foreach & foreachPartition

What is foreach?

foreach is an ACTION in PySpark that applies a function to each element of an RDD. Unlike map(), it does NOT return a new RDD — it returns nothing (Unit). It is used purely for SIDE EFFECTS: writing to databases, logging, calling external APIs, updating caches.

> **Real-World Analogy — Zomato Notification System**
> When Zomato marks an order as 'Delivered', it needs to send a notification to each customer. The list of delivered orders is an RDD. foreach() iterates over each order record and calls the notification API for that customer. The data is not transformed — just acted upon. That's the essence of foreach.

foreach Syntax

```
# Basic foreach — print each record (for debugging)
orders = sc.parallelize([
    ('ORD001', 'Rahul', 250.0),
    ('ORD002', 'Priya', 180.5),
    ('ORD003', 'Arjun', 420.0)
])

# Simple print (debugging only — don't use in production)
orders.foreach(lambda order: print(f'Processing: {order[0]}'))

# Writing to an external system
def send_to_kafka(record):
    from kafka import KafkaProducer
    producer = KafkaProducer(bootstrap_servers='localhost:9092')
    producer.send('orders-topic', str(record).encode())
    producer.close()

orders.foreach(send_to_kafka)  # ← Runs on executors, not driver
```

⚠ The Connection Problem with foreach

The code above has a CRITICAL performance problem. It opens a new Kafka connection for EVERY single record. For 10 million records = 10 million connections. This is catastrophically slow and kills the database/Kafka broker.

```
# ❌BAD: Opens 10 million DB connections for 10 million records
def insert_record(record):
    conn = mysql.connect(host='db', user='root', password='pass')  # NEW
connection each time!
    cursor = conn.cursor()
    cursor.execute('INSERT INTO orders VALUES (%s, %s, %s)', record)
    conn.commit()
    conn.close()  # Close each time

orders_rdd.foreach(insert_record)  # For 10M records → 10M connections!
```

✅foreachPartition — The Production Solution

foreachPartition gives your function an ITERATOR over an entire partition. You open ONE connection per partition, process all records in batch, then close. With 100 partitions of 100K records: 100 connections instead of 10 million.

```
# ✅GOOD: foreachPartition — one connection per partition
def insert_partition(records):
    import mysql.connector
    conn = mysql.connector.connect(host='db', user='root', password='pass')
    cursor = conn.cursor()
    batch = []
    for record in records:
        batch.append(record)
        if len(batch) >= 1000:          # Batch insert every 1000 records
            cursor.executemany('INSERT INTO orders VALUES (%s,%s,%s)', batch)
            conn.commit()
            batch = []
    if batch:                           # Insert remaining records
        cursor.executemany('INSERT INTO orders VALUES (%s,%s,%s)', batch)
        conn.commit()
    conn.close()                        # Close ONCE per partition

orders_rdd.foreachPartition(insert_partition)
# 100 partitions → 100 connections. 100x more efficient!
```

foreach vs foreachPartition Comparison

Aspect	foreach	foreachPartition
Function input	One element at a time	Iterator over entire partition
Connection overhead	1 connection per RECORD	1 connection per PARTITION
Batch operations	Not possible	Full batch insert support
Best use case	Simple logging, lightweight ops	DB inserts, Kafka publish, Redis write
Performance (10M records)	Extremely slow (10M connections)	Fast (100 connections)
Code complexity	Simpler	Slightly more complex

Sateesh Says

In every real-world data pipeline project — whether at Infosys, Wipro, or a startup — 99% of the time you should use foreachPartition, NOT foreach. The only time foreach is acceptable is for simple console logging during development. Any time you're connecting to a database, Kafka, Redis, or an external API in production, ALWAYS use foreachPartition. This is the single most impactful performance tip in this entire module.

✅Key Points to Remember

- foreach is an ACTION — iterates over elements for side effects, returns nothing
- foreach runs on EXECUTORS, not on the driver — variables modified inside are NOT seen on driver
- foreachPartition is 100x-1000x more efficient for DB/API operations
- Connection pool pattern: Open connection ONCE per partition, process all records, close ONCE
- Use Accumulator (not regular variable) to count records processed inside foreach
- Never use foreach for transformations — that's what map() is for

5. pipe Transformation

What is pipe?

pipe() is a transformation that passes each partition's data as standard input (stdin) to a shell command or external program. The output from the command (stdout) is captured and returned as a new RDD. Each partition runs the command as an independent process.

> **Real-World Analogy — Airtel Legacy Script Integration**
>
> Airtel has a 15-year-old Perl script that validates and formats mobile numbers (handles all Indian number portability rules). Rewriting this in PySpark would take 3 months. With pipe(), each Spark partition sends its data to the Perl script, gets validated numbers back, and continues the pipeline. Legacy system integration with zero rewriting.

How pipe() Works

```
# How data flows through pipe():
#
# Partition 0 data ──→ stdin  ──→ [shell command] ──→ stdout ──→ new RDD partition
0
# Partition 1 data ──→ stdin  ──→ [shell command] ──→ stdout ──→ new RDD partition
1
# Partition 2 data ──→ stdin  ──→ [shell command] ──→ stdout ──→ new RDD partition
2
#
# Each partition gets its own INDEPENDENT process
```

Syntax & Usage

```
# Example 1: Using a shell command to convert text to uppercase
data = sc.parallelize(['hello spark', 'pyspark is powerful', 'levelup training'])
upper = data.pipe('tr a-z A-Z')    # Shell command: tr (translate characters)
print(upper.collect())
# Output: ['HELLO SPARK', 'PYSPARK IS POWERFUL', 'LEVELUP TRAINING']

# Example 2: Using an AWK script for data transformation
transactions = sc.parallelize([
    '1001,5000,INR',
    '1002,100,USD',
    '1003,8000,INR'
])
# AWK script to extract INR transactions only
inr_only = transactions.pipe("awk -F',' '$3=="INR" {print $0}'")
print(inr_only.collect())
# Output: ['1001,5000,INR', '1003,8000,INR']

# Example 3: Pipe to an external Python script
# (Script file: /scripts/validate_pan.py)
validated = data.pipe('python3 /scripts/validate_pan.py')
```

When to Use pipe()

Scenario	pipe() appropriate?	Reason
Legacy Perl/Bash scripts for data transformation	✅Yes	Avoids rewriting tested code
Specialized C++ analytics tools	✅Yes	No Python API available
Simple string transformations	❌No — use map()	map() is faster, no process overhead
ML model inference	❌No — use mapPartitions()	Better memory management
Database operations	❌No — use foreachPartition()	Better connection management

Sateesh Says

pipe() is a niche but powerful feature. In enterprise projects at TCS or Capgemini, you'll often inherit legacy systems that have been running for 10-15 years. Nobody wants to rewrite them in PySpark from scratch. pipe() is your bridge — keep the legacy script running, integrate it into the modern Spark pipeline. It's like a translator between old and new worlds.

✅Key Points to Remember

- pipe() sends each partition to a shell command as stdin, captures stdout as new RDD
- Each partition runs as an INDEPENDENT process — fully parallel
- Best for integrating legacy shell scripts or external tools
- Each element is converted to string before being piped
- Process startup overhead per partition — not suitable for simple row-level operations
- Ensure the external script/command is available on ALL worker nodes

6. join Operations

What is join?

join() in PySpark RDD combines two pair RDDs based on matching keys — similar to SQL JOIN. Both RDDs must be key-value pairs (tuples). The result is a new RDD of (key, (value_from_left, value_from_right)) for all matching keys.

> **Real-World Analogy — Flipkart Customer + Order Merge**
> Flipkart's customer service team needs to show order history alongside customer details on one screen. Customers are in one database, orders in another. join() on customer_id combines them — just like a SQL JOIN on a relational database. The result has both customer name and order details in one record.

All Join Types with Code

```
# Setup: Customers and Orders data
customers = sc.parallelize([
    (1, 'Rahul'),
    (2, 'Priya'),
    (3, 'Arjun'),     # No orders
    (4, 'Meena')      # No orders
])
orders = sc.parallelize([
    (1, 1200.0),
    (1, 850.0),       # Rahul has 2 orders
    (2, 3500.0),
    (5, 200.0)        # No customer record for ID 5
])

# 1. INNER JOIN — only matching keys from BOTH
inner = customers.join(orders)
print(sorted(inner.collect()))
# [(1,('Rahul',1200.0)), (1,('Rahul',850.0)), (2,('Priya',3500.0))]
# Note: ID 3, 4 (no orders) and ID 5 (no customer) are ALL excluded

# 2. LEFT OUTER JOIN — all customers, None if no order
left = customers.leftOuterJoin(orders)
print(sorted(left.collect()))
# [(1,('Rahul',1200.0)), (1,('Rahul',850.0)),
#  (2,('Priya',3500.0)), (3,('Arjun',None)), (4,('Meena',None))]
# Note: ID 3 and 4 included with None. ID 5 excluded.

# 3. RIGHT OUTER JOIN — all orders, None if no customer
right = customers.rightOuterJoin(orders)
print(sorted(right.collect()))
# [(1,('Rahul',1200.0)), (1,('Rahul',850.0)),
#  (2,('Priya',3500.0)), (5,(None,200.0))]
# Note: ID 5 included. ID 3 and 4 excluded.

# 4. FULL OUTER JOIN — all records from both
full = customers.fullOuterJoin(orders)
print(sorted(full.collect()))
# All records from both RDDs. None wherever key is missing on either side.
```

```
# 5. subtractByKey — customers who NEVER placed an order
no_orders = customers.subtractByKey(orders)
print(no_orders.collect())
# [(3, 'Arjun'), (4, 'Meena')]
```

Join Types Quick Reference

Join Type	Keeps From Left	Keeps From Right	SQL Equivalent
join()	Only matching keys	Only matching keys	INNER JOIN
leftOuterJoin()	ALL records	Only matching keys (else None)	LEFT JOIN
rightOuterJoin()	Only matching keys (else None)	ALL records	RIGHT JOIN
fullOuterJoin()	ALL records (None if missing)	ALL records (None if missing)	FULL OUTER JOIN
subtractByKey()	Keys NOT in right RDD	Not used	LEFT ANTI JOIN

Performance: Broadcast Join (Critical for Production)

A standard join triggers a SHUFFLE — expensive. If one RDD is small (fits in executor memory), use Broadcast Join: broadcast the small RDD and do a map-side lookup. Zero shuffle!

```
# Problem: Standard join on 500MB customers + 200GB orders = massive shuffle
# Solution: Broadcast the small RDD (customers)

# Step 1: Collect small RDD to driver and broadcast
customers_bc = sc.broadcast(dict(customers.collect()))
# {'1': 'Rahul', '2': 'Priya', ...}  — broadcast to all executors

# Step 2: Enrich large RDD using map (NO shuffle!)
def enrich_order(order):
    order_id, cust_id, amount = order
    cust_name = customers_bc.value.get(cust_id, 'Unknown')
    return (order_id, cust_name, amount)

enriched_orders = orders_rdd.map(enrich_order)
# Reads from local broadcast copy — no network movement for 200GB orders!
```

> **Sateesh Says**
>
> *In 90% of real-world projects, one of the two datasets being joined is actually small — a lookup table, reference data, or a config file. ALWAYS check: 'Is one side small enough to broadcast?' If yes, broadcast it and save yourself from a massive shuffle. At Flipkart's scale (200GB orders + 2GB product catalog), broadcast join is 50x faster than standard join. This question comes up in almost every Spark interview.*

✅Key Points to Remember

- join() requires BOTH RDDs to be key-value pairs (tuples with key as first element)
- Default join() is INNER JOIN — only matching keys from both sides
- leftOuterJoin keeps ALL left records; missing right values become None
- join() triggers a SHUFFLE — one of the most expensive operations
- If one RDD is small: ALWAYS use Broadcast Join (collect + broadcast + map)
- subtractByKey() is perfect for finding 'records not in another dataset' (anti-join)

7. union

What is union?

union() is a transformation that combines two RDDs vertically — appending all elements from both RDDs into one. Both RDDs must have the same data type/structure. No key matching happens — it's purely stacking rows. Equivalent to SQL UNION ALL (duplicates are preserved).

> **Real-World Analogy — HDFC Multi-Region Data Consolidation**
> HDFC Bank runs separate processing for North, South, East, and West regions. At end of day, each region produces an RDD of settled transactions. union() combines all 4 regional RDDs into one national settlement RDD — all records preserved, nothing merged or matched. Like combining 4 stacks of printed reports into one pile.

Syntax & Usage

```
# Myntra: Consolidate orders from 4 warehouses
north_orders  = sc.parallelize([('P001', 'Jacket', 10), ('P002', 'Shoes', 5)])
south_orders  = sc.parallelize([('P001', 'Jacket', 8), ('P003', 'Dress', 12)])
east_orders   = sc.parallelize([('P004', 'Bag', 3)])
west_orders   = sc.parallelize([('P002', 'Shoes', 7), ('P005', 'Hat', 2)])

# Chain unions
all_orders =
north_orders.union(south_orders).union(east_orders).union(west_orders)
print(all_orders.count())   # 7 (all records, including 'Jacket' twice and 'Shoes'
twice)

# ✅Better: sc.union() for multiple RDDs (flat DAG, avoids StackOverflow for 100+
RDDs)
rdds = [north_orders, south_orders, east_orders, west_orders]
all_orders = sc.union(rdds)   # Flat union — always use for 10+ RDDs

# union keeps duplicates (like SQL UNION ALL)
# To get unique records: call .distinct() after union
unique_orders = all_orders.map(lambda x: x[0]).distinct()  # Unique product IDs
print(unique_orders.collect())  # ['P001', 'P002', 'P003', 'P004', 'P005']

# Count by product across all warehouses
total_stock = all_orders.map(lambda x: (x[0], x[2])).reduceByKey(lambda a,b: a+b)
print(sorted(total_stock.collect()))
# [('P001',18), ('P002',12), ('P003',12), ('P004',3), ('P005',2)]
```

union vs join — Key Difference

Aspect	union()	join()
Direction	Vertical — stacks rows	Horizontal — combines columns
Key matching	No — all rows kept	Yes — matched on key
Schema requirement	Same structure on both sides	Both must be key-value pairs

SQL equivalent	UNION ALL	JOIN
Duplicates	Preserved	Keys can repeat (one-to-many)
Shuffle	No (narrow transformation)	Yes (wide transformation)
Use case	Combine same-structure datasets	Enrich data using related datasets

Sateesh Says

Common mistake: Students confuse union and join. Remember it with this trick — union is like stacking sheets of paper (same format, more rows). join is like stapling related sheets together side-by-side (combining columns). Myntra combining 4 warehouse files = union. Myntra matching customer names to their orders = join.

✅Key Points to Remember

- union() is a NARROW transformation — no shuffle, very fast
- Both RDDs must have the same data structure/type
- Preserves duplicates — use .distinct() afterward if you want unique records
- For 10+ RDDs: use sc.union(list_of_rdds) to avoid deep DAG (StackOverflow risk)
- union() is equivalent to SQL UNION ALL (not UNION DISTINCT)
- Partition count of result = sum of partitions from both input RDDs

8. mapPartitions & mapPartitionsWithIndex

What is mapPartitions?

mapPartitions() is a transformation that processes one ENTIRE PARTITION at a time instead of one record at a time like map(). Your function receives an ITERATOR over all elements in a partition and must return an ITERATOR (using yield or return an iterable). This enables massive performance gains when operations have a fixed setup cost.

> **Real-World Analogy — PhonePe Fraud Scoring at Scale**
>
> Imagine PhonePe needs to score 50 million UPI transactions for fraud using a TensorFlow model (200MB). With map() — load the 200MB model 50 million times. With mapPartitions() — load the model ONCE per partition. If there are 500 partitions of 100K records each: 500 model loads total vs 50 million. That's a 100,000x reduction in model loading overhead.

map() vs mapPartitions() — The Core Difference

```
# map() — function called ONCE PER ELEMENT
def expensive_map(record):          # Called 10,000,000 times
    model = load_model()            # 10,000,000 model loads! Catastrophically
slow
    return model.predict(record)

result = rdd.map(expensive_map)     # DO NOT use for expensive setup

# ──────────────────────────────────────────────────────────────

# mapPartitions() — function called ONCE PER PARTITION
def efficient_mapPartitions(records):  # Called 100 times (for 100 partitions)
    model = load_model()               # Load ONCE per partition
    for record in records:             # Process all records with same model
        yield model.predict(record)    # yield makes it a generator (memory
efficient)

result = rdd.mapPartitions(efficient_mapPartitions)  # 100 model loads. Fast!
```

Practical Examples

Example 1: Database Batch Insert (Production Pattern)

```
# Swiggy: Insert 200M orders into Cassandra
def cassandra_batch_insert(records):
    from cassandra.cluster import Cluster
    cluster = Cluster(['cassandra-node1', 'cassandra-node2'])  # ONE connection
    session = cluster.connect('swiggy_orders')
    insert_stmt = session.prepare(
        'INSERT INTO orders (order_id, customer_id, amount) VALUES (?, ?, ?)'
    )
    batch = []
    for record in records:            # Process all records in partition
        batch.append(record)
        if len(batch) >= 500:
```

```
            for r in batch:
                session.execute(insert_stmt, r)
            batch = []
    for r in batch:                    # Flush remaining
        session.execute(insert_stmt, r)
    cluster.shutdown()                 # Close ONCE per partition
    return iter([])                    # mapPartitions must return iterator

orders_rdd.mapPartitions(cassandra_batch_insert).count()  # Action to trigger
```

Example 2: ML Model Scoring

```
# CRED: Score 40 million credit applications
def credit_score_partition(applications):
    import pickle
    # Load model ONCE per partition (not per record!)
    with open('/models/credit_model.pkl', 'rb') as f:
        model = pickle.load(f)
    for app_id, features in applications:
        score = model.predict_proba([features])[0][1]  # probability of default
        yield (app_id, round(score, 4))

applications_rdd = sc.parallelize(application_data, 200)  # 200 partitions
scores = applications_rdd.mapPartitions(credit_score_partition)
scores.saveAsTextFile('/data/credit_scores/2024-01-15')
```

mapPartitionsWithIndex

mapPartitionsWithIndex() is like mapPartitions() but passes the PARTITION INDEX as the first argument to your function. Use this when your processing logic depends on WHICH partition you're in.

```
# When to use mapPartitionsWithIndex:
# Processing 28 state-wise partitions where each state has different tax rules

state_configs = {
    0: {'state': 'Telangana', 'tax_rate': 0.18, 'holiday': '2024-01-14'},
    1: {'state': 'Maharashtra', 'tax_rate': 0.18, 'holiday': '2024-01-15'},
    # ... one config per partition
}

def process_state_data(partition_idx, records):
    config = state_configs[partition_idx]   # Look up state using partition index
    tax_rate = config['tax_rate']
    for txn_id, amount in records:
        tax = amount * tax_rate
        yield (txn_id, amount, tax, config['state'])

state_rdd = sc.parallelize(transactions, 28)  # 28 partitions = 28 states
result = state_rdd.mapPartitionsWithIndex(process_state_data)
```

Critical Rule: Use yield (Generators) NOT Lists

```
# ✗WRONG: Creates full list in memory (OOM for large partitions)
def bad_partition_func(records):
    result = []
```

```
    for r in records:
        result.append(process(r))    # Entire partition in memory!
    return result                    # Python list — all in RAM

# ✅CORRECT: Generator — lazy, processes one at a time
def good_partition_func(records):
    for r in records:
        yield process(r)             # Produces one item at a time — memory
efficient

# If function returns nothing but uses RDD for side effects:
def side_effect_func(records):
    for r in records:
        do_something(r)
    return iter([])                  # Must return empty iterator, not None
```

map vs mapPartitions vs foreachPartition — When to Use What

Operation	Input	Returns	Best Use Case
map()	One element	One element (new RDD)	Simple transformations (parse, format)
mapPartitions()	Iterator over partition	Iterator (new RDD)	Heavy setup: DB/model/connection needed
foreach()	One element	Nothing (side effect)	Simple logging, lightweight side effects
foreachPartition()	Iterator over partition	Nothing (side effect)	DB insert, Kafka publish, Redis write

Sateesh Says

mapPartitions is the secret weapon of every senior Spark developer. Map processes records like a factory where each worker opens a new toolbox for each item. mapPartitions processes like a worker who opens the toolbox ONCE at the start of their shift and uses it for every item. The toolbox = database connection, ML model, file handle, HTTP client. Any expensive setup = use mapPartitions. This is what separates a 3 LPA Spark developer from a 15 LPA one.

✅Key Points to Remember

- mapPartitions() processes entire PARTITION at once — setup cost amortized across all records
- Function must receive an iterator and return an iterator (use yield for lazy evaluation)
- mapPartitionsWithIndex() provides the partition number — use when processing logic is partition-specific
- ALWAYS use yield (generator) instead of building a list — prevents OOM on large partitions
- Return iter([]) if function has side effects and doesn't produce output
- Best for: ML model loading, DB connection setup, external API clients, file handles
- Pair with repartition() to control how many 'setups' happen (partitions = number of setups)

9. Interview Questions & Model Answers

Question	Model Answer
What happens if saveAsTextFile output directory exists?	Job FAILS with DirectoryAlreadyExistsException. Fix: delete first, use timestamped paths, or use DataFrame .write.mode('overwrite').
coalesce(200) on a 10-partition RDD — what is the result?	Still 10 partitions. coalesce() cannot increase — it silently ignores the request. Use repartition(200) to increase.
Why is repartition expensive?	It triggers a full shuffle — all data moves across the network to redistribute evenly. Involves disk I/O, serialization, and network transfer.
foreach vs foreachPartition — which is better for DB inserts?	foreachPartition — opens ONE DB connection per partition. foreach opens one per record. For 10M records: 100 connections vs 10M connections. Always use foreachPartition for I/O.
When would you use pipe() in a production pipeline?	When integrating with legacy shell scripts, Perl/AWK programs, or external tools that have no Python API. Avoids rewriting battle-tested code.
What is Broadcast Join and when should you use it?	When one RDD is small (fits in memory): collect it, broadcast it, do a map-side lookup. Eliminates shuffle for the large RDD. Use when one side is under 500MB.
union() vs join() — what's the key difference?	union() stacks rows vertically (SQL UNION ALL). join() combines columns horizontally based on matching keys (SQL JOIN). union = more rows, join = more columns.
mapPartitions vs map — when to prefer mapPartitions?	When setup is expensive (DB connection, model loading, file handle). Setup happens once per partition instead of once per record. Use yield for memory-efficient generators.
How do you prevent StackOverflow when union-ing 365 daily RDDs?	Use sc.union(list_of_rdds) — creates a flat DAG. Chaining .union() 365 times creates 365 levels of nested RDD references causing stack overflow during DAG analysis.
What is a narrow vs wide transformation? Give examples.	Narrow: output depends on ONE input partition (map, filter, flatMap, mapPartitions, union) — no shuffle. Wide: depends on MULTIPLE partitions (groupByKey, join, repartition) — triggers shuffle.

10. Module 6 — Quick Reference Card

Operation	Type	Shuffle?	Returns	Best Use Case
saveAsTextFile()	Action	No	Nothing (writes files)	Export RDD to HDFS/S3/local
coalesce(n)	Transformation	No	New RDD	Reduce partitions before write
repartition(n)	Transformation	Yes	New RDD	Increase/rebalance partitions
foreach()	Action	No	Nothing (side effect)	Logging, lightweight writes
foreachPartition()	Action	No	Nothing (side effect)	DB inserts, Kafka, Redis
pipe(cmd)	Transformation	No	New RDD	Legacy shell script integration
join()	Transformation	Yes	New RDD (key,(v1,v2))	Inner join on common key
leftOuterJoin()	Transformation	Yes	New RDD	Keep all left records
rightOuterJoin()	Transformation	Yes	New RDD	Keep all right records
fullOuterJoin()	Transformation	Yes	New RDD	Keep all records both sides
subtractByKey()	Transformation	Yes	New RDD	Anti-join / find missing
union()	Transformation	No	New RDD	Combine same-structure RDDs
mapPartitions()	Transformation	No	New RDD	Heavy setup per partition
mapPartitionsWithIndex()	Transformation	No	New RDD	Partition-aware processing

Sateesh Says

Print this Quick Reference Card and keep it on your desk. In every interview, coding test, or production debugging session, these 14 operations cover 80% of everything you need for PySpark RDD work. Master these — understand not just WHAT they do but WHY and WHEN — and you will clear any data engineering interview from TCS to Google.